SameSite=Lax

A SIMPLE GUIDE TO HOW IT WORKS

CONCEPTS | EXAMPLES | REAL-WORLD SCENARIOS



HAROON SHAHID

SAMESITE=LAX

Rule

A cookie configured with **SameSite=Lax** is sent with same-site requests when the cookie's other attributes and browser policies permit it.

For cross-site requests, the cookie is sent only when both conditions are satisfied:

- The request is a top-level navigation.
- The request uses a safe HTTP method, normally GET.

A top-level navigation can happen when the user clicks a link, JavaScript changes the page's location, or an HTML form is submitted.

Important: SameSite=Lax does not check whether the user personally clicked something. It checks the request's site context, whether it is a top-level navigation, and the HTTP method.

Safe HTTP Method

What does "safe HTTP method" mean?

In HTTP, a method is considered safe when its defined purpose is to retrieve information without requesting a change to the server's state.

Some safe methods include:

- `GET`
- `HEAD`
- `OPTIONS`

For your notes, focus mainly on `GET`, because it is the method normally used when navigating to a webpage through a link.

`POST`, `PUT`, and `DELETE` are not considered safe methods.

Same-site is not the same as same-origin

URL A	URL B	Same-site?
https://bank.com	https://bank.com/profile	Yes
https://app.example.com	https://evil.example.com	Yes
https://bank.com	https://evil.com	No
http://bank.com	https://bank.com	No, under schemeful SameSite rules

Modern SameSite calculations generally consider the URL scheme and registrable domain.

Important: Two different subdomains can be different origins but the same site.

An attacker-controlled sibling subdomain may therefore create same-site requests that SameSite restrictions alone do not block.

Example 1 — Normal Link Click

- You are logged into: **bank.com**
- The bank previously sent this response header to your browser: **Set-Cookie: sessionid=123; Path=/; Secure; HttpOnly; SameSite=Lax**
- Your browser stored the cookie with these attributes.
- Now you visit: **facebook.com**
- Someone posts a link: **https://bank.com/profile**
- You click it.

What happens?

Browser sends: **Cookie: sessionid=123**

So **bank.com** recognizes you.

You stay logged in.

Example 2 — CSRF Attack

- You are logged into: **bank.com**
- A hacker creates:

```
<form action="https://bank.com/transfer" method="POST">
  <input name="amount" value="1000">
  <input name="to" value="attacker">
</form>

<script>
document.forms[0].submit();
</script>
```

- And host it on **evil.com**
- You visit: **evil.com**
- The form automatically submits to: **bank.com**

What happens with SameSite=Lax?

The victim visits **evil.com**, and JavaScript automatically submits the HTML form to **bank.com**.

Because the form does not specify a **target** attribute, it uses the default value **_self**. Therefore, the submission normally causes a **top-level navigation to bank.com in the same tab**, replacing the current webpage.

But if the form specifies **target="_blank"**, the submission normally causes a **top-level navigation to bank.com in a new tab or window**, while the original webpage remains open, provided the browser does not block the new tab or window.

However, the request uses the POST method.

POST is not a safe HTTP method.

Therefore, the browser does not include the explicitly configured **SameSite=Lax** session cookie in this cross-site POST request.

Without the required session cookie, the bank cannot identify the victim's authenticated session through that cookie.

Assuming the endpoint requires this session for authentication, the transfer fails.

The cookie is excluded because this is a cross-site POST request, not because the form was submitted automatically or because the request was a background request.

How Cookie Behaves With Each Value

Attribute	Correct behavior
SameSite=Strict	The cookie is sent with same-site requests, but not cross-site requests, even when the user clicks a normal link from another site.
SameSite=Lax	The cookie is sent with same-site requests and qualifying cross-site top-level navigations using safe methods, normally GET.
SameSite=None	SameSite imposes no cross-site restriction, but the cookie must use **Secure**, and other cookie attributes and browser privacy restrictions still apply.

Important: SameSite controls when a cookie can be sent in relation to the initiating site. It does not remove the cookie's Domain, Path, or other restrictions.

It means SameSite is only one of several rules that control whether a browser sends a cookie.

For example, even if **SameSite=Lax** allows a cookie:

- Domain: The cookie must belong to the requested domain.
- Path: The requested URL path must match the cookie's Path restriction.
- Secure: If present, the cookie normally requires HTTPS.

In simple words: Even if SameSite allows a cookie, the browser must still check the cookie's other restrictions before sending it.

Question 1

What if this form is on the same site?

```
<form action="https://bank.com/transfer" method="POST">
  <input name="amount" value="1000">
  <input name="to" value="attacker">
</form>

<script>
document.forms[0].submit();
</script>
```

And the page itself is also on: **https://bank.com**

Answer

Then the request is **same-site**, not cross-site.

The browser sends the cookie normally.

bank.com ---> bank.com

Cookie sent? **YES**

SameSite controls cookie restrictions based on the request's site context. In this example, the request is same-site, so SameSite=Lax does not prevent the cookie from being sent, even though the request uses POST.

Example

You are logged into: **bank.com**

A page on bank.com contains:

```
<form action="/transfer" method="POST">
...
</form>
```

When submitted: **bank.com ---> bank.com**

Cookie is sent.

This is normal website behavior.

SameSite=Lax allows cookies in same-site POST requests, provided the cookie's other attributes and browser policies permit them.

SameSite does not determine whether the action itself is authorized.

If an attacker somehow controls a page or script within the target application's trusted site, SameSite alone may not prevent unwanted requests.

Question 2

What do you mean by "top-level navigation"?

Answer

Top-level navigation means opening a webpage in the main area of a browser tab or window, either by replacing the current webpage or opening it in a new tab or window, rather than loading it inside an iframe (nested frame) or making a background request.

A top-level navigation can happen through:

- Clicking a normal link.
- JavaScript changing **window.location**.
- Submitting an HTML form, either manually or automatically.
- Opening a page in a new top-level browsing context, such as a new tab.

A top-level navigation does not require the user to click anything.

For SameSite=Lax, a cross-site top-level navigation must also use a safe HTTP method, normally GET, for the cookie to be included.

Direct example

This JavaScript automatically initiates a cross-site top-level GET navigation:

```
<script>
    window.location = "https://bank.com/profile";
</script>
```

Even though the user didn't click anything, an applicable **SameSite=Lax** cookie can accompany the navigation.

Example 1

```
<a href="https://bank.com/profile">
    My Profile
</a>
```

You click it.

The entire page changes to: **https://bank.com/profile**

This is **top-level navigation**.

Example 2

You are on: **facebook.com**

Someone posts:

```
<a href="https://bank.com">
    Visit Bank
```

```
</a>
```

You click it.

Browser loads: **bank.com** in the whole tab.

This is top-level navigation.

With **SameSite=Lax**, the cookie **IS SENT**.

Top-Level Navigation vs. Background Requests

Example 1: Automatically submitted POST form

```
<form action="https://bank.com/transfer" method="POST">
    <input name="amount" value="1000">
</form>

<script>
    document.forms[0].submit();
</script>
```

This normally causes a top-level navigation to **bank.com**.

However, because the request is cross-site and uses POST, the browser does not include an explicitly configured **SameSite=Lax** session cookie.

Example 2: Automatically submitted GET form

```
<form action="https://bank.com/profile" method="GET">
</form>

<script>
    document.forms[0].submit();
</script>
```

This also normally causes a top-level navigation.

Because it uses GET, an applicable **SameSite=Lax** cookie can be sent.

Example 3: Fetch request

```
fetch("https://bank.com/profile");
```

This is a background request, not a top-level navigation.

Fetch Credentials vs. SameSite

When JavaScript makes a request using **fetch()**, the browser also considers the Fetch API's **credentials** setting.

By default, **fetch()** uses:

credentials: "same-origin"

This means credentials such as cookies are included by default only for requests to the same origin.

For example, suppose JavaScript running on **https://evil.com** executes:

```
fetch("https://bank.com/profile");
```

This is a cross-origin request.

The browser does not include the bank's cookies by default because Fetch uses **credentials: "same-origin"**.

What if we explicitly allow cross-origin credentials?

An attacker could write:

```
fetch("https://bank.com/profile", {
    credentials: "include"
});
```

The **include** setting tells Fetch to allow credentials on cross-origin requests.

This means credentials such as cookies are included for requests to the same and cross-origin.

However, this does not override the cookie's SameSite restrictions.

Because this is a cross-site Fetch request rather than a top-level navigation, an explicitly configured **SameSite=Lax** cookie is still excluded.

The important difference

- **Fetch credentials setting:** Determines whether Fetch permits credentials to accompany a request.
- **SameSite attribute:** Determines whether a cookie is eligible to be sent based on the request's site context and navigation rules.

Both restrictions must allow the cookie for it to be included.

Remember: **credentials: "include"** does not bypass **SameSite=Lax**.

Also, **credentials: "include"** does not automatically allow JavaScript to read the cross-origin response. CORS rules still control access to the response.

Request	Lax cookie sent?
Same-site, cross-origin Fetch with default **credentials: "same-origin"**	No, because of the Fetch credentials setting
Same-site, cross-origin Fetch with **credentials: "include"**	Yes, if the cookie is otherwise eligible

---

Example 4: Image request

<img src="https://bank.com/profile">

This request loads an image from another website without opening that website in the main browser tab. Therefore, it is not a top-level navigation.

Example 5: Request inside an iframe

<iframe src="https://bank.com/profile"></iframe>

This navigates a nested browsing context rather than the top-level page.

An explicitly configured **SameSite=Lax** cookie is normally excluded from these cross-site background or nested requests.

Background requests do not always block cookies. If a background request is same-site, the browser can send **SameSite=Lax** cookies, as long as the other cookie rules and request settings allow it.

Summary: When Does SameSite=Lax Send Cookies?

Assume the cookie is explicitly set to **SameSite=Lax**, belongs to the target website, and is otherwise eligible to be sent.

Request	Lax cookie sent?
Same-site GET	Yes
Same-site POST	Yes
Cross-site link click causing top-level GET navigation	Yes
Cross-site JavaScript causing top-level GET navigation	Yes
Cross-site automatically submitted GET form causing top-level navigation	Yes
Cross-site manually submitted POST form	No
Cross-site automatically submitted POST form	No
Cross-site image request	No
Cross-site iframe navigation	No
Cross-site Fetch request	No

The rule is not whether the user clicked something.

For cross-site requests, ask:

Is this a top-level navigation AND does it use a safe HTTP method?

If both conditions are satisfied, the Lax restriction permits the cookie.

If either condition is not satisfied, the Lax restriction excludes the cookie.

All results assume the cookie's other attributes and browser policies permit transmission. For Fetch, the credentials setting imposes an additional requirement.

Question 3:

How Does the Browser Decide Whether to Send a SameSite=Lax Cookie?

The browser does not simply check whether the user clicked a link or JavaScript initiated the request.

It evaluates the request's context.

For an explicitly configured **SameSite=Lax** cookie, the important questions are:

First: Is the request same-site or cross-site?

If it is same-site, the SameSite=Lax restriction permits the cookie, including for POST requests.

If it is cross-site, continue to the next question.

Second: Is the request a top-level navigation?

A normal link, a JavaScript location change, or an HTML form submission can all cause top-level navigation.

A Fetch request, image request, or iframe navigation does not normally qualify.

Third: Does the top-level navigation use a safe HTTP method?

If yes, such as GET, SameSite=Lax permits the cookie.

If no, such as POST, SameSite=Lax blocks the cookie.

Example A:

```
window.location = "https://bank.com/profile";
```

This causes a cross-site, top-level GET navigation.

The Lax restriction permits the cookie.

Example B:

```
<form action="https://bank.com/transfer" method="POST">
   <input name="amount" value="1000">
</form>
```

```
<script>
    document.forms[0].submit();
</script>
```

This causes a cross-site, top-level POST navigation.

The Lax restriction blocks the cookie because POST is not a safe method.

Example C:

```
fetch("https://bank.com/profile", {
    credentials: "include"
});
```

This is a cross-site background request, not a top-level navigation.

Even though the request explicitly asks the browser to include credentials, **SameSite=Lax** prevents the cookie from being sent cross-site in this context.

The browser considers other cookie restrictions in addition to SameSite.

Explicit Lax vs. Lax-by-default

This is particularly relevant to PortSwigger's SameSite bypass labs.

Lax-by-default exception

Some browsers apply Lax restrictions to cookies that do not explicitly specify a SameSite attribute.

However, certain browsers use a more permissive default behavior that may allow recently set cookies to accompany cross-site top-level POST requests for approximately two minutes.

This exception does not apply in the same way to cookies explicitly configured with **SameSite=Lax**.

Therefore, **SameSite=Lax** and an omitted SameSite attribute should not always be treated as identical.

Schemeless Same-Site and Schemeful Same-Site

There are two distinct contexts for calculating "same-site": Schemeless Same-Site and Schemeful Same-Site.

However, in modern web security, **they have officially merged into a single standard**. The web standards body (W3C/IETF) and major browsers updated the formal definition so that "site" intrinsically includes the scheme.

Today, the original "schemeless" context is considered an obsolete, legacy behavior. [Same-Site]